

جلسه چهارم: حلقه‌ها در پایتون (نسخه کامل و جامع)

در این جلسه به صورت گام‌به‌گام و کامل با حلقه‌ها در پایتون آشنا می‌شویم. حلقه‌ها زمانی استفاده می‌شوند که بخواهیم یک قطعه کد را چندین بار اجرا کنیم یا روی یک دنباله (لیست، رشته، تاپل، دیکشنری و ...) پیمایش داشته باشیم.

اهداف یادگیری

- درک تفاوت حلقه‌های `for` و `while`
- آشنایی با `range`، `enumerate`، `zip` و پیمایش دیکشنری‌ها
- تسلط بر دستورات کنترل جریان: `break`، `continue` و `pass` + بخش `else` در حلقه‌ها
- کار با حلقه‌های تو در تو (Nested Loops) و الگوهای رایج
- نکات مهم کارایی و خوانایی
- تمرین‌های متنوع (از آسان تا پیشرفته) + تمرین‌های ترکیبی (حلقه + شرط)

پیشنهاد: هر مثال را اجرا کنید و خروجی را ببینید، سپس آن را تغییر بدهید و دوباره اجرا کنید تا رفتار حلقه‌ها را بهتر درک کنید.

حلقه چیست؟

- حلقه قطعه کدی است که چندبار تکرار می‌شود.
- در پایتون عمدتاً دو نوع حلقه داریم:
 - `for`: برای پیمایش روی یک دنباله یا هر شیء قابل تکرار (iterable)
 - `while`: تا زمانی که یک شرط برقرار باشد، اجرا می‌شود

حلقه `for`

حلقه `for` برای پیمایش روی آیتم‌های یک دنباله استفاده می‌شود. شکل کلی:

```
for item in iterable
    # بدنه حلقه
```

مثال‌های پایه

```
In [1]: # پیمایش روی یک لیست
fruits = ["apple", "banana", "cherry"]
for f in fruits:
    print(f)
```

apple
banana
cherry

```
In [2]: # پیمایش روی رشته (هر کاراکتر یک آیتم است)
s = "python"
for ch in s:
    print(ch, end=" ")
```

p y t h o n

تابع range

تابع range دنباله‌ای از اعداد تولید می‌کند (تنها هنگام پیمایش؛ حافظه‌پسند):

- range(n) → از 0 تا n-1
- range(start, stop) → از start تا stop-1
- range(start, stop, step) → با گام step (می‌تواند منفی باشد)

نکته: معمولاً نیازی نیست range را به لیست تبدیل کنید. فقط زمانی تبدیل کنید که واقعاً به لیست نیاز دارید.

```
In [3]: # مثال‌های range
print(list(range(5)))           # 0..4
print(list(range(2, 7)))        # 2..6
print(list(range(10, 0, -2)))   # 10, 8, 6, 4, 2
```

```
[0, 1, 2, 3, 4]
[2, 3, 4, 5, 6]
[10, 8, 6, 4, 2]
```

ابزارهای کاربردی در حلقه‌ها

enumerate

وقتی به اندیس و مقدار همزمان نیاز دارید:

```
In [4]: names = ["Ali", "Sara", "Mehdi"]
for idx, name in enumerate(names, start=1):
    print(idx, name)
```

```
1 Ali
2 Sara
3 Mehdi
```

zip

برای پیمایش همزمان روی چند دنباله با طول‌های مشابه:

```
In [5]: first_names = ["Ali", "Sara", "Mehdi"]
        scores = [18, 20, 17]
        for fn, sc in zip(first_names, scores):
            print(f"{fn}: {sc}")
```

```
Ali: 18
Sara: 20
Mehdi: 17
```

پیمایش روی دیکشنری

- روی کلیدها: `dict.keys()` یا `dict.keys()`
- روی مقادیر: `dict.values()`
- روی جفت‌های کلید/مقدار: `dict.items()`

```
In [6]: person = {"name": "Sara", "age": 23, "city": "Tehran"}

        print("Keys:")
        for k in person:
            print(k)

        print("\nValues:")
        for v in person.values():
            print(v)

        print("\nItems:")
        for k, v in person.items():
            print(k, "=>", v)
```

```
Keys:
name
age
city
```

```
Values:
Sara
23
Tehran
```

```
Items:
name => Sara
age => 23
city => Tehran
```

حلقه while

وقتی تا زمان برقرار بودن یک شرط نیاز به تکرار داریم از `while` استفاده می‌کنیم.

مراقب باشید شرط را جایی در بدنه به روز کنید تا حلقه بی‌نهایت نشود.

```
In [7]: x = 0
while x < 5:
    print("x =", x)
    x += 1  # اگر این خط حذف شود، حلقه بی‌نهایت می‌شود
```

```
x = 0
x = 1
x = 2
x = 3
x = 4
```

continue، break و pass + بخش else در حلقه‌ها

- break: خروج فوری از حلقه
- continue: پرش به تکرار بعدی
- pass: انجام هیچ‌کاری (برای جای خالی/اسکلت کد)
- else در حلقه‌ها: وقتی حلقه بدون break تمام شود، بخش else اجرا می‌شود.

مثال‌ها

```
In [8]: # break: یافتن اولین عددی که بر 7 بخش‌پذیر است
for i in range(1, 50):
    if i % 7 == 0:
        print("Found:", i)
        break
```

Found: 7

```
In [9]: # continue: چاپ فقط اعداد فرد
for i in range(10):
    if i % 2 == 0:
        continue
    print(i)
```

```
1
3
5
7
9
```

```
In [10]: # حلقه: بررسی اول بودن عدد else
n = 29
is_prime = True
for d in range(2, int(n**0.5) + 1):
    if n % d == 0:
        is_prime = False
        break
else:
    # نخوردیم به اینجا می‌رسیم break فقط اگر
    pass

print(n, "prime?" , is_prime)
```

29 prime? True

حلقه‌های تو در تو (Nested Loops)

گاهی لازم است درون یک حلقه، حلقه دیگری داشته باشیم؛ مانند کار با **ماتریس‌ها**، چاپ **الگوها** یا مقایسه تمام زوج‌های عناصر. الگوی کلی:

```
for i in outer_iterable
    for j in inner_iterable
        # کار با i و j
```

نکات مهم

- پیچیدگی زمانی معمولاً ضرب می‌شود (مثلاً $O(n*m)$ یا $O(n^2)$)، پس از نظر کارایی مراقب باشید.
- از شکستن زودهنگام (early break) استفاده کنید تا بیهوده ادامه ندهید.
- پایتون **break** برچسب‌دار ندارد؛ برای خروج از چند سطح حلقه می‌توانید:
 - منطق را در تابع قرار دهید و با **return** خارج شوید.
 - از پرچم/فلگ استفاده کنید.
 - یا از استثنا (با احتیاط) برای کنترل جریان استفاده کنید.

مثال ۱: جدول ضرب فرمت‌شده (۱۰×۱۰)

```
In [11]: # جدول ضرب 1 تا 10 با قالب‌بندی ستونی
width = 4
# سطر عنوان
print(" " * width, end="")
for j in range(1, 11):
    print(f"{j:>{width}}", end="")
print()
print("-" * (width * 11))

for i in range(1, 11):
    print(f"{i:>{width-1}}|", end="")
    for j in range(1, 11):
        print(f"{i*j:>{width}}", end="")
    print()
```



```
In [14]: matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9],
    ]

# جمع عناصر
total = 0
for row in matrix:
    for val in row:
        total += val
print("Sum =", total)

# ترانپوز (Transpose)
transpose = []
for j in range(len(matrix[0])):
    col = []
    for i in range(len(matrix)):
        col.append(matrix[i][j])
    transpose.append(col)

print("Transpose =", transpose)
```

Sum = 45

Transpose = [[1, 4, 7], [2, 5, 8], [3, 6, 9]]

45

Transpose = [[1, 4, 7], [2, 5, 8], [3, 6, 9]]

مثال ۴: یافتن تمام زوج اعداد با مجموع هدف (Nested) + بهینه سازی

```
In [15]: nums = [2, 7, 11, 15, -2, 9]
target = 9

pairs = []
for i in range(len(nums)):
    for j in range(i+1, len(nums)): # جلوگیری از تکرار زوج های معکوس
        if nums[i] + nums[j] == target:
            pairs.append((nums[i], nums[j]))

print("All pairs (nested):", pairs)

# راه بهینه تر با دیکشنری/مجموعه (O(n))
seen = set()
pairs_fast = []
for x in nums:
    need = target - x
    if need in seen:
        pairs_fast.append((need, x))
    seen.add(x)

print("All pairs (set-based):", pairs_fast)
```

All pairs (nested): [(2, 7), (11, -2)]

All pairs (set-based): [(2, 7), (11, -2)]

فهرست‌سازی فشرده (List Comprehension) با حلقه‌های تو در تو

گاهی می‌توانید حلقه‌های تو در تو را در یک عبارت فشرده کنید؛ ولی خوانایی را فدا نکنید.

```
In [16]: # لیست لیست‌ها (Flatten) تخت کردن
nested = [[1,2,3],[4,5],[6]]
flat = [x for row in nested for x in row]
print(flat)

# با شرط (i, j) تولید زوج‌های مرتب
pairs = [(i, j) for i in range(3) for j in range(3) if i != j]
print(pairs)
```

```
[1, 2, 3, 4, 5, 6]
[(0, 1), (0, 2), (1, 0), (1, 2), (2, 0), (2, 1)]
```

نکات کارایی و خوانایی

- اگر امکان دارد به جای $O(n^2)$ از ساختارهای داده مناسب (مثل مجموعه و دیکشنری) استفاده کنید.
- برای خروج زود هنگام از حلقه‌ها (وقتی پاسخ پیدا شد) از `break` یا `return` در تابع کمک بگیرید.
- نام‌گذاری متغیرها (مثل `row`, `col`, `i`, `j`) را معنادار انتخاب کنید.
- از `enumerate` به جای شمارنده دستی استفاده کنید.
- بخش `else` در حلقه را بشناسید؛ خوانایی را بالا می‌برد (مثال بررسی اول بودن).

تمرین‌ها

بخش A — پایه

1. با استفاده از حلقه `for` اعداد ۱ تا ۲۰ را چاپ کنید.
2. مجموع اعداد ۱ تا ۱۰۰ را با `for` محاسبه کنید.
3. از کاربر یک عدد بگیرید و جدول ضرب آن را تا ۱۰ چاپ کنید.
4. یک رشته بگیرید و تعداد حروف صدادار انگلیسی آن (`a, e, i, o, u`) را بشمارید.
5. تعداد کاراکترهای رقمی موجود در یک رشته را بشمارید.

بخش B — حلقه + شرط

1. تمام اعداد بین ۱ تا ۵۰ که بر ۳ بخش پذیرند را چاپ کنید.
2. اعداد اول بین ۱ تا ۱۰۰ را چاپ کنید (از `break` یا `else` استفاده کنید).
3. از کاربر ۱۰ عدد بگیرید و بزرگ‌ترین آن‌ها را بیابید.

4. مثلثی از ستاره‌ها با ارتفاع ورودی کاربر چاپ کنید (راست‌گرا یا هرم).
5. مسأله **FizzBuzz**: برای ۱ تا ۱۰۰، اگر بر ۳ بخش‌پذیر بود چاپ **Fizz**، اگر بر ۵ بخش‌پذیر بود چاپ **Buzz**، اگر بر هر دو بخش‌پذیر بود **FizzBuzz**، در غیر اینصورت خود عدد را چاپ کنید.

بخش C — حلقه‌های تو در تو

- جدول ضرب ۱۲×۱۲ را به صورت مرتب چاپ کنید.
- الگویی از ستاره‌ها به شکل **لوزی** چاپ کنید.
- یک ماتریس ۳×۳ از ورودی بگیرید و **ترانهاده‌ی** آن را تولید کنید.
- لیست دوبعدی‌ای از اعداد داده‌شده را **تخت** کنید (بدون استفاده از کتابخانه).
- تمام **زوج‌های یکتا** از آرایه‌ای داده‌شده که مجموعشان برابر مقدار هدف است را چاپ کنید (از تکرار $(a,b)/(b,a)$ جلوگیری کنید).

بخش D — پیشرفته (ترکیب حلقه + شرط)

- تولید مثلث پاسکال با ارتفاع دلخواه کاربر.
- یافتن اعداد **آرمسترانگ** بین ۱ تا ۱۰۰۰.
- یافتن اعداد **کامل** (Perfect Numbers) در یک بازه.
- چاپ **مارپیچ** (Spiral) در یک ماتریس $n \times n$ با اعداد $۱..n^2$.
- ترانهاده درجا** (بدون ماتریس جدید) برای ماتریس مربعی.

پیشنهاد: برای هر تمرین، نمونه ورودی/خروجی کوچک طراحی کنید و سپس کدتان را تست کنید.

مثال تکمیلی: جست‌وجوی عدد در ماتریس (Break از چند سطح با تابع)

```
In [17]: def find_number(matrix, target):
    for i, row in enumerate(matrix):
        for j, val in enumerate(row):
            if val == target:
                return i, j # خروج فوری از هر دو حلقه
    return None

m = [
    [10, 11, 12],
    [20, 21, 22],
    [30, 31, 32],
]

print(find_number(m, 22)) # (1, 2)
print(find_number(m, 99)) # None
```

(1, 2)
None

جمع‌بندی

- `for` برای پیمایش روی دنباله‌ها و `while` برای تکرار وابسته به شرط است.
- `enumerate` ، `range` و `zip` ابزارهای کلیدی برای حلقه‌های تمیز و قابل‌خواندن هستند.
- `break` و `continue` کنترل دقیق جریان را در اختیار ما می‌گذارند؛ `else` در حلقه‌ها را فراموش نکنید.
- حلقه‌های تو در تو قدرتمندند اما از نظر کارایی هزینه دارند؛ تا می‌توانید از ساختارهای داده مناسب استفاده کنید.

🌟 موفق باشید!